

Skill Session 11

Task 1

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session11_history.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session11_history.c -o session11_history  
j_rakshitha@RAKSHITHA:~$ ./session11_history  
Session 11 - Command History  
History capacity: 5 commands  
  
history-shell$ ls  
  
history-shell$ pwd  
  
history-shell$ echo hello  
  
history-shell$ date  
  
history-shell$ whoami  
  
history-shell$ history  
  
Command History  
-----  
1  ls  
2  pwd  
3  echo hello  
4  date  
5  whoami  
  
history-shell$ get 2  
History 2: pwd  
  
history-shell$ get 5  
History 5: whoami  
  
history-shell$ check  
History consistency: VALID  
  
history-shell$ exit  
  
Final history:  
  
Command History  
-----  
1  ls  
2  pwd  
3  echo hello
```

```
4 date
5 whoami
j_rakshitha@RAKSHITHA:~$ ./session11_history
Session 11 - Command History
History capacity: 5 commands

history-shell$ one

history-shell$ two

history-shell$ four

history-shell$ six

history-shell$ five

history-shell$ seven
History buffer full. Cannot store: seven

history-shell$ exit

Final history:

Command History
-----
1 one
2 two
3 four
4 six
5 five
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HISTORY_SIZE 5
#define COMMAND_SIZE 100

typedef struct
{
    char commands[HISTORY_SIZE][COMMAND_SIZE];
    int count;
} History;

void initialize_history(History *history)
{
    history->count = 0;

    for (int i = 0; i < HISTORY_SIZE; i++)
    {
        history->commands[i][0] = '\0';
    }
}

int add_command(History *history, const char *command)
{
    if (history->count >= HISTORY_SIZE)
    {
        printf("History buffer full. Cannot store: %s\n", command);
        return 0;
    }

    strncpy(history->commands[history->count],
            command,
            COMMAND_SIZE - 1);

    history->commands[history->count][COMMAND_SIZE - 1] = '\0';

    history->count++;

    return 1;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_history.c *

void display_history(const History *history)
{
    printf("\nCommand History\n");
    printf("-----\n");

    if (history->count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    for (int i = 0; i < history->count; i++)
    {
        printf("%d %s\n", i + 1, history->commands[i]);
    }
}

void retrieve_command(const History *history, int number)
{
    if (number < 1 || number > history->count)
    {
        printf("Invalid history number: %d\n", number);
        return;
    }

    printf("History %d: %s\n",
           number,
           history->commands[number - 1]);
}

int validate_history(const History *history)
{
    if (history->count < 0 || history->count > HISTORY_SIZE)
    {
        return 0;
    }

    for (int i = 0; i < history->count; i++)
    {
        if (history->commands[i][0] == '\0')
        |
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_history.c *
    return 0;
}
}
return 1;
}
int main()
{
    History history;

    initialize_history(&history);

    char command[COMMAND_SIZE];

    printf("Session 11 - Command History\n");
    printf("History capacity: %d commands\n", HISTORY_SIZE);

    while (1)
    {
        printf("\nhistory-shell$ ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            display_history(&history);
            continue;
        }
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste     ^J Justify  ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_history.c *
{
    break;
}

if (strcmp(command, "history") == 0)
{
    display_history(&history);
    continue;
}

if (strncmp(command, "get ", 4) == 0)
{
    int number = atoi(command + 4);
    retrieve_command(&history, number);
    continue;
}

if (strcmp(command, "check") == 0)
{
    if (validate_history(&history))
        printf("History consistency: VALID\n");
    else
        printf("History consistency: INVALID\n");

    continue;
}

if (strlen(command) == 0)
{
    continue;
}

add_command(&history, command);
}

printf("\nFinal history:\n");
display_history(&history);

return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Task 2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

j_rakshitha@RAKSHITHA:~$ nano session11_pipeline.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session11_pipeline.c -o session11_pipeline
j_rakshitha@RAKSHITHA:~$ ./session11_pipeline
Session 11 - Pipeline Structure

Pipeline Layout
-----
Process 1: ls -l -->
Process 2: grep .c -->
Process 3: sort

Pipeline validation: VALID

Execution order:
1. ls -l
2. grep .c
3. sort
j_rakshitha@RAKSHITHA:~$ |

```

```

GNU nano 8.7.1 session11_pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20

typedef struct
{
    char command[100];
    char *args[MAX_ARGS];
} Command;

typedef struct
{
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

void initialize_pipeline(Pipeline *pipeline)
{
    pipeline->count = 0;
}

int add_command(Pipeline *pipeline, const char *command)
{
    if (pipeline->count >= MAX_COMMANDS)
    {
        printf("Pipeline capacity reached.\n");
        return 0;
    }

    strncpy(pipeline->commands[pipeline->count].command,
            command,
            sizeof(pipeline->commands[pipeline->count].command) - 1);

    pipeline->commands[pipeline->count].command[sizeof(pipeline->commands[pipeline->count].command) - 1] = '\0';

    pipeline->count++;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_pipeline.c *
    return 1;
}

void display_pipeline(const Pipeline *pipeline)
{
    printf("\nPipeline Layout\n");
    printf("-----\n");

    if (pipeline->count == 0)
    {
        printf("Pipeline is empty.\n");
        return;
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("Process %d: %s", i + 1,
            pipeline->commands[i].command);

        if (i < pipeline->count - 1)
            printf(" --> ");

        printf("\n");
    }
}

int validate_pipeline(const Pipeline *pipeline)
{
    if (pipeline->count < 1 ||
        pipeline->count > MAX_COMMANDS)
    {
        return 0;
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        if (strlen(pipeline->commands[i].command) == 0)
            return 0;
    }

    return 1;
}

| return 1;

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session11_pipeline.c *

    for (int i = 0; i < pipeline->count; i++)
    {
        if (strlen(pipeline->commands[i].command) == 0)
            return 0;
    }

    return 1;
}

int main()
{
    Pipeline pipeline;

    initialize_pipeline(&pipeline);

    printf("Session 11 - Pipeline Structure\n");

    add_command(&pipeline, "ls -l");
    add_command(&pipeline, "grep .c");
    add_command(&pipeline, "sort");

    display_pipeline(&pipeline);

    if (validate_pipeline(&pipeline))
        printf("\nPipeline validation: VALID\n");
    else
        printf("\nPipeline validation: INVALID\n");

    printf("\nExecution order:\n");

    for (int i = 0; i < pipeline.count; i++)
    {
        printf("%d. %s\n",
                i + 1,
                pipeline.commands[i].command);
    }

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Skill Session 12

Task 1

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

j_rakshitha@RAKSHITHA:~$ nano session12_pipe.c

j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session12_pipe.c -o session12_pipe
j_rakshitha@RAKSHITHA:~$ ./session12_pipe
Session 12 - Pipe Communication
-rwxr-xr-x 1 j_rakshitha j_rakshitha 16360 Aug  3 05:51 command_executor
-rw-r--r-- 1 j_rakshitha j_rakshitha  981 Aug  3 05:47 command_executor.c
drwxr-xr-x 5 j_rakshitha j_rakshitha 4096 Aug 18 05:21 ossp_project
-rw-r--r-- 1 j_rakshitha j_rakshitha 1970 Sep 17 04:42 session10_export.c
-rw-r--r-- 1 j_rakshitha j_rakshitha  963 Sep 17 04:40 session10_pwd_exit.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2997 Sep 17 05:10 session11_history.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2232 Sep 17 05:22 session11_pipeline.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1560 Sep 17 05:24 session12_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha  391 Aug 12 06:39 task1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha  933 Aug 12 06:41 task2.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1218 Aug 12 06:44 task3.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3814 Aug 12 06:50 task4.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1786 Aug 12 06:52 task5.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1378 Aug 18 05:48 week4_T1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha  828 Aug 18 05:45 week4_T2.c
Producer and consumer processes completed.
Pipe communication completed successfully.
j_rakshitha@RAKSHITHA:~$ |

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session12_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Session 12 - Pipe Communication\n");

    /*
     * First child:
     * Executes ls -l and sends its output into the pipe.
     */
    pid_t producer = fork();

    if (producer == -1)
    {
        perror("fork");
        return 1;
    }

    if (producer == 0)
    {
        close(pipefd[0]);

        if (dup2(pipefd[1], STDOUT_FILENO) == -1)
        {
            perror("dup2");
            exit(1);
        }

        close(pipefd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);

        perror("execlp ls");
        exit(1);
    }

    /*
     * Second child:
     * Reads from the pipe and executes grep ".c".
     */

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session12_pipe.c *
    execlp("ls", "ls", "-l", (char *)NULL);

    perror("execlp ls");
    exit(1);
}

/*
 * Second child:
 * Reads from the pipe and executes grep ".c".
 */
pid_t consumer = fork();

if (consumer == -1)
{
    perror("fork");
    return 1;
}

if (consumer == 0)
{
    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2");
        exit(1);
    }

    close(pipefd[0]);

    execlp("grep", "grep", ".c", (char *)NULL);

    perror("execlp grep");
    exit(1);
}

/*
 * Parent does not use the pipe.
 */
close(pipefd[0]);
close(pipefd[1]);

waitpid(producer, NULL, 0);
waitpid(consumer, NULL, 0);

printf("Producer and consumer processes completed.\n");
printf("Pipe communication completed successfully.\n");

return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

Task 2

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
j_rakshitha@RAKSHITHA:~$ nano session12_multi_pipe.c

j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session12_multi_pipe.c -o session12_multi_pipe
j_rakshitha@RAKSHITHA:~$ ./session12_multi_pipe
Session 12 - Multiple Pipe Pipeline
Pipeline: ls -l | grep .c | sort

-rw-r--r-- 1 j_rakshitha j_rakshitha 391 Aug 12 06:39 task1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 828 Aug 18 05:45 week4_T2.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 933 Aug 12 06:41 task2.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 963 Sep 17 04:40 session10_pwd_exit.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 981 Aug 3 05:47 command_executor.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1218 Aug 12 06:44 task3.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1378 Aug 18 05:48 week4_T1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1560 Sep 17 05:24 session12_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1786 Aug 12 06:52 task5.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1970 Sep 17 04:42 session10_export.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2232 Sep 17 05:22 session11_pipeline.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2576 Sep 17 05:27 session12_multi_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2997 Sep 17 05:10 session11_history.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3814 Aug 12 06:50 task4.c
-rwxr-xr-x 1 j_rakshitha j_rakshitha 16360 Aug 3 05:51 command_executor
drwxr-xr-x 5 j_rakshitha j_rakshitha 4096 Aug 18 05:21 ossp_project

All pipeline processes completed.
All pipe descriptors cleaned up.
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session12_multi_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_PROCESSES 3
#define NUM_PIPES 2

int main()
{
    int pipes[NUM_PIPES][2];
    pid_t pids[NUM_PROCESSES];

    /*
     * Create all required pipes.
     */
    for (int i = 0; i < NUM_PIPES; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    printf("Session 12 - Multiple Pipe Pipeline\n");
    printf("Pipeline: ls -l | grep .c | sort\n\n");

    /*
     * Process 1: ls -l
     */
    pids[0] = fork();

    if (pids[0] == -1)
    {
        perror("fork");
        return 1;
    }

    if (pids[0] == 0)
    {
        /*
         * stdout -> pipe 0
         */
        dup2(pipes[0][1], STDOUT_FILENO);

        /*
         * Close all pipe descriptors.
         */
        for (int i = 0; i < NUM_PIPES; i++)
        {
            ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
            ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line M-E Redo
        }
    }
}
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session12_multi_pipe.c *
    close(pipes[i][0]);
    close(pipes[i][1]);
}

execlp("ls", "ls", "-l", (char *)NULL);

perror("execlp ls");
exit(1);
}

/*
 * Process 2: grep .c
 */
pids[1] = fork();

if (pids[1] == -1)
{
    perror("fork");
    return 1;
}

if (pids[1] == 0)
{
    /*
     * stdin <- pipe 0
     */
    dup2(pipes[0][0], STDIN_FILENO);

    /*
     * stdout -> pipe 1
     */
    dup2(pipes[1][1], STDOUT_FILENO);

    for (int i = 0; i < NUM_PIPES; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    execlp("grep", "grep", ".c", (char *)NULL);

    perror("execlp grep");
    exit(1);
}

/*
 * Process 3: sort
 */
pids[2] = fork();
| if (pids[2] == -1)

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

j_rakshitha@RAKSHITHA: ~ × + ▾

GNU nano 8.7.1session12_multi_pipe.c *

```
    /*
    pids[2] = fork();

    if (pids[2] == -1)
    {
        perror("fork");
        return 1;
    }

    if (pids[2] == 0)
    {
        /*
        * stdin <- pipe 1
        */
        dup2(pipes[1][0], STDIN_FILENO);

        for (int i = 0; i < NUM_PIPES; i++)
        {
            close(pipes[i][0]);
            close(pipes[i][1]);
        }

        execlp("sort", "sort", (char *)NULL);

        perror("execlp sort");
        exit(1);
    }

    /*
    * Parent closes all pipe descriptors.
    */
    for (int i = 0; i < NUM_PIPES; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /*
    * Wait for all processes.
    */
    for (int i = 0; i < NUM_PROCESSES; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("\nAll pipeline processes completed.\n");
    printf("All pipe descriptors cleaned up.\n");

    return 0;
}

```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

^C Location

^_ Go To Line

M-U Undo

M-E Redo